```java
import java.util.*;

/**
 * ============================================================
 *  TOP 100 LEETCODE PROBLEMS — JAVA SOLUTIONS
 *  Categories: Arrays | Strings | Linked Lists
 * ============================================================
 *
 *  ARRAYS  (Problems  1 - 40)
 *  STRINGS (Problems 41 - 75)
 *  LINKED LISTS (Problems 76 - 100)
 *
 *  Every solution includes:
 *   - Problem statement summary
 *   - Approach explanation
 *   - Time & Space complexity
 * ============================================================
 */
public class Top100LeetCode {

    // ============================================================
    // ==================== HELPER CLASSES ========================
    // ============================================================

    /** Singly Linked List Node */
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
        ListNode(int val, ListNode next) { this.val = val; this.next = next; }
    }

    // ============================================================
    // ======================== ARRAYS ============================
    // ============================================================

    // ------------------------------------------------------------
    // 1. TWO SUM
    // Given an array of integers, return indices of two numbers
    // that add up to a target value.
    // Approach: HashMap — store complement → index.
    // Time: O(n) | Space: O(n)
    // ------------------------------------------------------------
    public int[] twoSum(int[] nums, int target) {
```

```
    Map<Integer, Integer> map = new HashMap<>(); // value → index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}

// ------------------------------------------------------------
// 2. BEST TIME TO BUY AND SELL STOCK (LC #121)
// Find the max profit from one buy-sell transaction.
// Approach: Track running minimum price; compute profit at each step.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) {
            minPrice = price;           // new cheapest buy day
        } else {
            maxProfit = Math.max(maxProfit, price - minPrice); // sell today?
        }
    }
    return maxProfit;
}

// ------------------------------------------------------------
// 3. CONTAINS DUPLICATE (LC #217)
// Return true if any value appears at least twice.
// Approach: HashSet — O(1) lookup per element.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        if (!seen.add(num)) return true; // add() returns false if already pr
    }
    return false;
}

// ------------------------------------------------------------
// 4. PRODUCT OF ARRAY EXCEPT SELF (LC #238)
// Return array where output[i] = product of all elements except nums[i].
```

```
// Approach: Two-pass prefix/suffix products without using division.
// Time: O(n) | Space: O(1) extra (output array doesn't count)
// ------------------------------------------------------
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    // Pass 1: result[i] = product of all elements to the LEFT of i
    result[0] = 1;
    for (int i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }
    // Pass 2: multiply by running suffix product from the RIGHT
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }
    return result;
}

// ------------------------------------------------------
// 5. MAXIMUM SUBARRAY (LC #53) — Kadane's Algorithm
// Find contiguous subarray with the largest sum.
// Approach: At each index decide: extend current subarray or start fresh.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int maxSubArray(int[] nums) {
    int currentSum = nums[0];
    int maxSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        // Starting fresh (nums[i]) vs extending (currentSum + nums[i])
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}

// ------------------------------------------------------
// 6. MAXIMUM PRODUCT SUBARRAY (LC #152)
// Find contiguous subarray with the largest product.
// Approach: Track both max AND min (negatives can flip sign).
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int maxProduct(int[] nums) {
    int maxProd = nums[0], minProd = nums[0], result = nums[0];
    for (int i = 1; i < nums.length; i++) {
        // A negative number swaps max and min
```

```
        if (nums[i] < 0) { int tmp = maxProd; maxProd = minProd; minProd = tm
        maxProd = Math.max(nums[i], maxProd * nums[i]);
        minProd = Math.min(nums[i], minProd * nums[i]);
        result = Math.max(result, maxProd);
    }
    return result;
}

// ------------------------------------------------------------
// 7. FIND MINIMUM IN ROTATED SORTED ARRAY (LC #153)
// Array was sorted then rotated; find minimum in O(log n).
// Approach: Binary search — if mid > right, minimum is in right half.
// Time: O(log n) | Space: O(1)
// ------------------------------------------------------------
public int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) {
            lo = mid + 1; // rotation point is in the right half
        } else {
            hi = mid;     // minimum is in the left half (inclusive)
        }
    }
    return nums[lo];
}

// ------------------------------------------------------------
// 8. SEARCH IN ROTATED SORTED ARRAY (LC #33)
// Search for target in a rotated sorted array.
// Approach: Modified binary search — determine which half is sorted.
// Time: O(log n) | Space: O(1)
// ------------------------------------------------------------
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        // Left half is sorted
        if (nums[lo] <= nums[mid]) {
            if (target >= nums[lo] && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // Right half is sorted
            if (target > nums[mid] && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
```

```
    return -1;
}

// ------------------------------------------------------
// 9. THREE SUM (LC #15)
// Find all unique triplets that sum to zero.
// Approach: Sort + two pointers. Fix one element, two-pointer the rest.
// Time: O(n²) | Space: O(1) extra
// ------------------------------------------------------
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicates
        int lo = i + 1, hi = nums.length - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                lo++; hi--;
            } else if (sum < 0) lo++;
            else hi--;
        }
    }
    return result;
}

// ------------------------------------------------------
// 10. CONTAINER WITH MOST WATER (LC #11)
// Find two lines that form a container holding the most water.
// Approach: Two pointers — always move the shorter line inward.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int maxArea(int[] height) {
    int lo = 0, hi = height.length - 1, maxWater = 0;
    while (lo < hi) {
        int water = Math.min(height[lo], height[hi]) * (hi - lo);
        maxWater = Math.max(maxWater, water);
        // Move the pointer with the shorter line
        if (height[lo] < height[hi]) lo++;
        else hi--;
    }
    return maxWater;
}
```

```java
// ----------------------------------------------------------
// 11. TRAPPING RAIN WATER (LC #42)
// Compute how much water is trapped between elevation bars.
// Approach: Two pointers track maxLeft/maxRight seen so far.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public int trap(int[] height) {
    int lo = 0, hi = height.length - 1;
    int maxLeft = 0, maxRight = 0, water = 0;
    while (lo < hi) {
        if (height[lo] <= height[hi]) {
            if (height[lo] >= maxLeft) maxLeft = height[lo];
            else water += maxLeft - height[lo]; // trapped above this bar
            lo++;
        } else {
            if (height[hi] >= maxRight) maxRight = height[hi];
            else water += maxRight - height[hi];
            hi--;
        }
    }
    return water;
}

// ----------------------------------------------------------
// 12. MOVE ZEROES (LC #283)
// Move all zeros to end while maintaining relative order of non-zeros.
// Approach: Two-pointer in-place — write pointer tracks insert position.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public void moveZeroes(int[] nums) {
    int write = 0; // next position to write a non-zero
    for (int num : nums) {
        if (num != 0) nums[write++] = num;
    }
    while (write < nums.length) nums[write++] = 0; // fill tail with zeros
}

// ----------------------------------------------------------
// 13. REMOVE DUPLICATES FROM SORTED ARRAY (LC #26)
// Remove duplicates in-place; return count of unique elements.
// Approach: Slow/fast pointer — slow advances only on new unique value.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
```

```
        if (nums[fast] != nums[slow]) {
            nums[++slow] = nums[fast]; // place next unique value
        }
    }
    return slow + 1;
}

// ------------------------------------------------------------
// 14. ROTATE ARRAY (LC #189)
// Rotate array to the right by k steps.
// Approach: Reverse entire array, then reverse first k, then rest.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public void rotate(int[] nums, int k) {
    k %= nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}
private void reverse(int[] nums, int lo, int hi) {
    while (lo < hi) { int t = nums[lo]; nums[lo++] = nums[hi]; nums[hi--] = t
}

// ------------------------------------------------------------
// 15. MERGE SORTED ARRAY (LC #88)
// Merge nums2 into nums1 (which has enough space) in sorted order.
// Approach: Fill from the back to avoid overwriting unread elements.
// Time: O(m+n) | Space: O(1)
// ------------------------------------------------------------
public void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (i >= 0 && j >= 0) {
        nums1[k--] = (nums1[i] >= nums2[j]) ? nums1[i--] : nums2[j--];
    }
    while (j >= 0) nums1[k--] = nums2[j--]; // leftover from nums2
}

// ------------------------------------------------------------
// 16. FIND DUPLICATE NUMBER (LC #287)
// Array contains n+1 integers in [1,n]; find the duplicate.
// Approach: Floyd's cycle detection (treat array as linked list).
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int findDuplicate(int[] nums) {
    int slow = nums[0], fast = nums[0];
    // Phase 1: detect cycle
    do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast);
```

```
    // Phase 2: find entry point of cycle = duplicate
    slow = nums[0];
    while (slow != fast) { slow = nums[slow]; fast = nums[fast]; }
    return slow;
}

// ------------------------------------------------------
// 17. MISSING NUMBER (LC #268)
// Find missing number in [0, n].
// Approach: XOR all indices and values — duplicates cancel out.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int missingNumber(int[] nums) {
    int xor = nums.length; // start with n
    for (int i = 0; i < nums.length; i++) xor ^= i ^ nums[i];
    return xor;
}

// ------------------------------------------------------
// 18. SINGLE NUMBER (LC #136)
// Every element appears twice except one; find it.
// Approach: XOR all elements — pairs cancel (a^a=0), single remains.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) result ^= num;
    return result;
}

// ------------------------------------------------------
// 19. MAJORITY ELEMENT (LC #169) — Boyer-Moore Voting
// Find element that appears more than n/2 times.
// Approach: Cancel out different elements; survivor is majority.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int majorityElement(int[] nums) {
    int candidate = nums[0], count = 1;
    for (int i = 1; i < nums.length; i++) {
        if (count == 0) { candidate = nums[i]; count = 1; }
        else count += (nums[i] == candidate) ? 1 : -1;
    }
    return candidate;
}

// ------------------------------------------------------
// 20. SORT COLORS (LC #75) — Dutch National Flag
```

```
// Sort array of 0s, 1s, 2s in-place in one pass.
// Approach: Three pointers: low (0 boundary), mid (current), high (2 boundar
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public void sortColors(int[] nums) {
    int lo = 0, mid = 0, hi = nums.length - 1;
    while (mid <= hi) {
        if (nums[mid] == 0) {
            swap(nums, lo++, mid++); // 0 goes to front
        } else if (nums[mid] == 1) {
            mid++;                   // 1 is in place
        } else {
            swap(nums, mid, hi--);   // 2 goes to back (don't advance mid)
        }
    }
}
private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] =

// ------------------------------------------------------------
// 21. MERGE INTERVALS (LC #56)
// Merge overlapping intervals.
// Approach: Sort by start, then greedily merge.
// Time: O(n log n) | Space: O(n)
// ------------------------------------------------------------
public int[][] mergeIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    for (int[] interval : intervals) {
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0
            merged.add(interval); // no overlap — add new
        } else {
            merged.get(merged.size() - 1)[1] =
                Math.max(merged.get(merged.size() - 1)[1], interval[1]); // e
        }
    }
    return merged.toArray(new int[0][]);
}

// ------------------------------------------------------------
// 22. INSERT INTERVAL (LC #57)
// Insert a new interval into a sorted non-overlapping list.
// Approach: Three phases: before, overlap, after.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0, n = intervals.length;
```

```
    // Phase 1: intervals that end before new one starts
    while (i < n && intervals[i][1] < newInterval[0]) result.add(intervals[i+
    // Phase 2: merge overlapping
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);
    // Phase 3: intervals that start after new one ends
    while (i < n) result.add(intervals[i++]);
    return result.toArray(new int[0][]);
}

// ------------------------------------------------------------
// 23. SPIRAL MATRIX (LC #54)
// Return elements of matrix in spiral order.
// Approach: Shrink boundaries (top, bottom, left, right) after each pass.
// Time: O(m*n) | Space: O(1) extra
// ------------------------------------------------------------
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) result.add(matrix[top][c]);   //
        top++;
        for (int r = top; r <= bottom; r++) result.add(matrix[r][right]); //
        right--;
        if (top <= bottom)
            for (int c = right; c >= left; c--) result.add(matrix[bottom][c])
        bottom--;
        if (left <= right)
            for (int r = bottom; r >= top; r--) result.add(matrix[r][left]);
        left++;
    }
    return result;
}

// ------------------------------------------------------------
// 24. ROTATE IMAGE (LC #48)
// Rotate n×n matrix 90° clockwise in-place.
// Approach: Transpose then reverse each row.
// Time: O(n²) | Space: O(1)
// ------------------------------------------------------------
public void rotateImage(int[][] matrix) {
    int n = matrix.length;
```

```
    // Step 1: Transpose (swap matrix[i][j] with matrix[j][i])
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int t = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] =
        }
    // Step 2: Reverse each row
    for (int[] row : matrix) {
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int t = row[l]; row[l] = row[r]; row[r] = t;
        }
    }
}

// ------------------------------------------------------------
// 25. SET MATRIX ZEROES (LC #73)
// If any cell is 0, set its entire row and column to 0.
// Approach: Use first row/column as markers to avoid extra space.
// Time: O(m*n) | Space: O(1)
// ------------------------------------------------------------
public void setZeroes(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    boolean firstRowZero = false, firstColZero = false;
    // Check if first row/col should be zeroed
    for (int j = 0; j < n; j++) if (matrix[0][j] == 0) firstRowZero = true;
    for (int i = 0; i < m; i++) if (matrix[i][0] == 0) firstColZero = true;
    // Use first row/col as flags
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][j] == 0) { matrix[i][0] = 0; matrix[0][j] = 0; }
    // Zero out based on flags
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
    if (firstRowZero) Arrays.fill(matrix[0], 0);
    if (firstColZero) for (int i = 0; i < m; i++) matrix[i][0] = 0;
}

// ------------------------------------------------------------
// 26. SUBARRAY SUM EQUALS K (LC #560)
// Count subarrays whose sum equals k.
// Approach: Prefix sum + HashMap. If prefixSum[j] - prefixSum[i] = k,
//           then subarray [i+1..j] sums to k.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty prefix
```

```
    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num;
        count += prefixCount.getOrDefault(sum - k, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return count;
}

// ------------------------------------------------------------
// 27. LONGEST CONSECUTIVE SEQUENCE (LC #128)
// Find length of longest consecutive elements sequence.
// Approach: HashSet — only start counting from sequence start (num-1 absent)
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) set.add(num);
    int best = 0;
    for (int num : set) {
        if (!set.contains(num - 1)) { // start of a sequence
            int len = 1;
            while (set.contains(num + len)) len++;
            best = Math.max(best, len);
        }
    }
    return best;
}

// ------------------------------------------------------------
// 28. JUMP GAME (LC #55)
// Can you reach the last index? Each element is max jump length.
// Approach: Track furthest reachable index (greedy).
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public boolean canJump(int[] nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false; // current index is unreachable
        maxReach = Math.max(maxReach, i + nums[i]);
    }
    return true;
}

// ------------------------------------------------------------
// 29. JUMP GAME II (LC #45)
// Minimum number of jumps to reach the last index.
```

```java
// Approach: Greedy BFS levels — at each level expand to furthest reach.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int jump(int[] nums) {
    int jumps = 0, curEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == curEnd) { // must jump here
            jumps++;
            curEnd = farthest;
        }
    }
    return jumps;
}

// ------------------------------------------------------------
// 30. FIND ALL DUPLICATES IN ARRAY (LC #442)
// Numbers in [1,n]; find all that appear twice.
// Approach: Negate value at index num-1 as visited marker.
// Time: O(n) | Space: O(1) extra
// ------------------------------------------------------------
public List<Integer> findAllDuplicates(int[] nums) {
    List<Integer> result = new ArrayList<>();
    for (int num : nums) {
        int idx = Math.abs(num) - 1;
        if (nums[idx] < 0) result.add(idx + 1); // visited twice
        else nums[idx] = -nums[idx];             // mark visited
    }
    return result;
}

// ------------------------------------------------------------
// 31. NEXT PERMUTATION (LC #31)
// Rearrange numbers into the lexicographically next greater permutation.
// Approach: Find rightmost ascending pair, swap with next larger, reverse ta
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public void nextPermutation(int[] nums) {
    int n = nums.length, i = n - 2;
    // Step 1: Find rightmost nums[i] < nums[i+1]
    while (i >= 0 && nums[i] >= nums[i + 1]) i--;
    if (i >= 0) {
        // Step 2: Find rightmost element greater than nums[i]
        int j = n - 1;
        while (nums[j] <= nums[i]) j--;
        swap(nums, i, j);
    }
```

```
    // Step 3: Reverse suffix
    reverse(nums, i + 1, n - 1);
}

// ------------------------------------------------------------
// 32. MINIMUM SIZE SUBARRAY SUM (LC #209)
// Find shortest contiguous subarray with sum >= target.
// Approach: Sliding window — expand right, shrink left when sum >= target.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int minSubArrayLen(int target, int[] nums) {
    int lo = 0, sum = 0, minLen = Integer.MAX_VALUE;
    for (int hi = 0; hi < nums.length; hi++) {
        sum += nums[hi];
        while (sum >= target) {
            minLen = Math.min(minLen, hi - lo + 1);
            sum -= nums[lo++]; // shrink window from left
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}

// ------------------------------------------------------------
// 33. SLIDING WINDOW MAXIMUM (LC #239)
// Max in every window of size k.
// Approach: Monotonic deque — front always holds index of window max.
// Time: O(n) | Space: O(k)
// ------------------------------------------------------------
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < n; i++) {
        // Remove indices outside window
        if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
        // Remove smaller elements from back (they'll never be max)
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[dq.peekFirst()];
    }
    return result;
}

// ------------------------------------------------------------
// 34. 4SUM (LC #18)
// Find all unique quadruplets that sum to target.
// Approach: Fix two elements with nested loops + two-pointer inner search.
```

```java
// Time: O(n³) | Space: O(1) extra
// ------------------------------------------------------------
public List<List<Integer>> fourSum(int[] nums, int target) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    int n = nums.length;
    for (int i = 0; i < n - 3; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        for (int j = i + 1; j < n - 2; j++) {
            if (j > i + 1 && nums[j] == nums[j - 1]) continue;
            int lo = j + 1, hi = n - 1;
            while (lo < hi) {
                long sum = (long) nums[i] + nums[j] + nums[lo] + nums[hi];
                if (sum == target) {
                    result.add(Arrays.asList(nums[i], nums[j], nums[lo], nums
                    while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                    while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                    lo++; hi--;
                } else if (sum < target) lo++;
                else hi--;
            }
        }
    }
    return result;
}

// ------------------------------------------------------------
// 35. KOKO EATING BANANAS (LC #875)
// Find minimum eating speed to finish all piles in h hours.
// Approach: Binary search on speed [1, max(piles)].
// Time: O(n log m) where m = max pile | Space: O(1)
// ------------------------------------------------------------
public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = Arrays.stream(piles).max().getAsInt();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        int hours = 0;
        for (int pile : piles) hours += (pile + mid - 1) / mid; // ceil divis
        if (hours <= h) hi = mid; // can eat slower
        else lo = mid + 1;
    }
    return lo;
}

// ------------------------------------------------------------
// 36. FIRST MISSING POSITIVE (LC #41)
// Find smallest missing positive integer.
```

```java
// Approach: Use array itself as index map — place num at index num-1.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    // Place each num in its correct bucket if 1 <= num <= n
    for (int i = 0; i < n; i++) {
        while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
            swap(nums, i, nums[i] - 1);
        }
    }
    // First position where nums[i] != i+1 is the answer
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) return i + 1;
    }
    return n + 1;
}

// ------------------------------------------------------------
// 37. TOP K FREQUENT ELEMENTS (LC #347)
// Return k most frequent elements.
// Approach: HashMap + bucket sort by frequency.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) freq.merge(num, 1, Integer::sum);
    // Bucket: index = frequency, value = list of nums with that freq
    List<Integer>[] buckets = new List[nums.length + 1];
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        int f = e.getValue();
        if (buckets[f] == null) buckets[f] = new ArrayList<>();
        buckets[f].add(e.getKey());
    }
    int[] result = new int[k];
    int idx = 0;
    for (int f = buckets.length - 1; f >= 0 && idx < k; f--) {
        if (buckets[f] != null)
            for (int num : buckets[f]) if (idx < k) result[idx++] = num;
    }
    return result;
}

// ------------------------------------------------------------
// 38. ENCODE AND DECODE STRINGS (LC #271)
// Encode list of strings to single string; decode back.
// Approach: Length-prefix encoding: "<len>#<str>".
```

```java
// Time: O(n) encode/decode | Space: O(n)
// ------------------------------------------------------
public String encode(List<String> strs) {
    StringBuilder sb = new StringBuilder();
    for (String s : strs) sb.append(s.length()).append('#').append(s);
    return sb.toString();
}
public List<String> decode(String s) {
    List<String> result = new ArrayList<>();
    int i = 0;
    while (i < s.length()) {
        int j = s.indexOf('#', i);
        int len = Integer.parseInt(s.substring(i, j));
        result.add(s.substring(j + 1, j + 1 + len));
        i = j + 1 + len;
    }
    return result;
}

// ------------------------------------------------------
// 39. TWO SUM II — SORTED ARRAY (LC #167)
// Find two numbers in sorted array that add to target.
// Approach: Two pointers from both ends.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int[] twoSumII(int[] numbers, int target) {
    int lo = 0, hi = numbers.length - 1;
    while (lo < hi) {
        int sum = numbers[lo] + numbers[hi];
        if (sum == target) return new int[]{lo + 1, hi + 1}; // 1-indexed
        else if (sum < target) lo++;
        else hi--;
    }
    return new int[]{};
}

// ------------------------------------------------------
// 40. FIND PEAK ELEMENT (LC #162)
// A peak is greater than its neighbors. Return any peak index.
// Approach: Binary search — if mid < mid+1, peak is to the right.
// Time: O(log n) | Space: O(1)
// ------------------------------------------------------
public int findPeakElement(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] < nums[mid + 1]) lo = mid + 1; // ascending → peak righ
```

```
        else hi = mid;                                  // descending → peak le
    }
    return lo;
}

// ============================================================
// ====================== STRINGS =========================
// ============================================================

// ------------------------------------------------------------
// 41. VALID ANAGRAM (LC #242)
// Check if two strings are anagrams of each other.
// Approach: Count character frequencies (26-char array for lowercase).
// Time: O(n) | Space: O(1) — alphabet is constant size
// ------------------------------------------------------------
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    for (char c : t.toCharArray()) { if (--count[c - 'a'] < 0) return false;
    return true;
}

// ------------------------------------------------------------
// 42. GROUP ANAGRAMS (LC #49)
// Group array of strings into anagram families.
// Approach: Sort each word as key in HashMap.
// Time: O(n * k log k) where k = max word length | Space: O(n*k)
// ------------------------------------------------------------
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] chars = s.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}

// ------------------------------------------------------------
// 43. LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS (LC #3)
// Find length of longest substring with all unique characters.
// Approach: Sliding window + HashMap tracking last seen index.
// Time: O(n) | Space: O(min(n, alphabet))
// ------------------------------------------------------------
public int lengthOfLongestSubstring(String s) {
```

```
    Map<Character, Integer> lastSeen = new HashMap<>();
    int maxLen = 0, lo = 0;
    for (int hi = 0; hi < s.length(); hi++) {
        char c = s.charAt(hi);
        if (lastSeen.containsKey(c)) {
            lo = Math.max(lo, lastSeen.get(c) + 1); // shrink window past rep
        }
        lastSeen.put(c, hi);
        maxLen = Math.max(maxLen, hi - lo + 1);
    }
    return maxLen;
}

// ------------------------------------------------------------
// 44. LONGEST REPEATING CHARACTER REPLACEMENT (LC #424)
// Replace at most k characters; find longest substring of same char.
// Approach: Sliding window — window valid if (size - maxCount) <= k.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int lo = 0, maxCount = 0, maxLen = 0;
    for (int hi = 0; hi < s.length(); hi++) {
        maxCount = Math.max(maxCount, ++count[s.charAt(hi) - 'A']);
        // Replacements needed = window size - most frequent char count
        while (hi - lo + 1 - maxCount > k) count[s.charAt(lo++) - 'A']--;
        maxLen = Math.max(maxLen, hi - lo + 1);
    }
    return maxLen;
}

// ------------------------------------------------------------
// 45. MINIMUM WINDOW SUBSTRING (LC #76)
// Smallest window in s containing all characters of t.
// Approach: Two-pointer + frequency map + "formed" counter.
// Time: O(|s| + |t|) | Space: O(|s| + |t|)
// ------------------------------------------------------------
public String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);
    int required = need.size(), formed = 0;
    Map<Character, Integer> window = new HashMap<>();
    int lo = 0, minLen = Integer.MAX_VALUE, startIdx = 0;
    for (int hi = 0; hi < s.length(); hi++) {
        char c = s.charAt(hi);
        window.merge(c, 1, Integer::sum);
```

```
        if (need.containsKey(c) && window.get(c).equals(need.get(c))) formed+
        while (formed == required) {
            if (hi - lo + 1 < minLen) { minLen = hi - lo + 1; startIdx = lo;
            char left = s.charAt(lo++);
            window.merge(left, -1, Integer::sum);
            if (need.containsKey(left) && window.get(left) < need.get(left))
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(startIdx, startIdx
}

// ------------------------------------------------------------
// 46. VALID PALINDROME (LC #125)
// Check if string is palindrome ignoring non-alphanumeric chars.
// Approach: Two pointers skip non-alnum, compare chars case-insensitively.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public boolean isPalindrome(String s) {
    int lo = 0, hi = s.length() - 1;
    while (lo < hi) {
        while (lo < hi && !Character.isLetterOrDigit(s.charAt(lo))) lo++;
        while (lo < hi && !Character.isLetterOrDigit(s.charAt(hi))) hi--;
        if (Character.toLowerCase(s.charAt(lo)) != Character.toLowerCase(s.ch
            return false;
        lo++; hi--;
    }
    return true;
}

// ------------------------------------------------------------
// 47. LONGEST PALINDROMIC SUBSTRING (LC #5)
// Find the longest palindromic substring.
// Approach: Expand around center for each character (and gap).
// Time: O(n²) | Space: O(1)
// ------------------------------------------------------------
public String longestPalindrome(String s) {
    if (s.length() < 2) return s;
    int start = 0, maxLen = 1;
    for (int i = 0; i < s.length(); i++) {
        // Odd length palindromes
        int len1 = expandAroundCenter(s, i, i);
        // Even length palindromes
        int len2 = expandAroundCenter(s, i, i + 1);
        int len = Math.max(len1, len2);
        if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
```

```
        }
    }
    return s.substring(start, start + maxLen);
}
private int expandAroundCenter(String s, int lo, int hi) {
    while (lo >= 0 && hi < s.length() && s.charAt(lo) == s.charAt(hi)) { lo--
    return hi - lo - 1; // length of palindrome
}

// ------------------------------------------------------------
// 48. PALINDROMIC SUBSTRINGS (LC #647)
// Count all palindromic substrings.
// Approach: Same expand-around-center; count instead of tracking max.
// Time: O(n²) | Space: O(1)
// ------------------------------------------------------------
public int countSubstrings(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        count += countPalindromesFrom(s, i, i);     // odd
        count += countPalindromesFrom(s, i, i + 1); // even
    }
    return count;
}
private int countPalindromesFrom(String s, int lo, int hi) {
    int count = 0;
    while (lo >= 0 && hi < s.length() && s.charAt(lo--) == s.charAt(hi++)) co
    return count;
}

// ------------------------------------------------------------
// 49. VALID PARENTHESES (LC #20)
// Check if bracket string is valid (properly opened and closed).
// Approach: Stack — push open brackets, pop on matching close.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public boolean isValidParentheses(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') stack.push(c);
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == '}' && top != '{') return false;
            if (c == ']' && top != '[') return false;
        }
    }
```

```
    return stack.isEmpty();
}

// ------------------------------------------------------
// 50. GENERATE PARENTHESES (LC #22)
// Generate all valid combinations of n pairs of parentheses.
// Approach: Backtracking — add '(' if open < n, ')' if close < open.
// Time: O(4^n / sqrt(n)) Catalan number | Space: O(n)
// ------------------------------------------------------
public List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    generateParenHelper(result, new StringBuilder(), 0, 0, n);
    return result;
}
private void generateParenHelper(List<String> res, StringBuilder sb, int open
    if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
    if (open < n) {
        sb.append('(');
        generateParenHelper(res, sb, open + 1, close, n);
        sb.deleteCharAt(sb.length() - 1);
    }
    if (close < open) {
        sb.append(')');
        generateParenHelper(res, sb, open, close + 1, n);
        sb.deleteCharAt(sb.length() - 1);
    }
}

// ------------------------------------------------------
// 51. REVERSE WORDS IN A STRING (LC #151)
// Reverse the order of words (remove extra spaces).
// Approach: Split, trim, join in reverse.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------
public String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(' ');
    }
    return sb.toString();
}

// ------------------------------------------------------
// 52. STRING TO INTEGER ATOI (LC #8)
// Implement atoi with overflow handling, sign, whitespace.
```

```
// Approach: Step through: skip spaces → sign → digits → clamp.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int myAtoi(String s) {
    int i = 0, n = s.length(), sign = 1;
    long result = 0;
    while (i < n && s.charAt(i) == ' ') i++;            // skip spaces
    if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-'))
        sign = (s.charAt(i++) == '-') ? -1 : 1;         // sign
    while (i < n && Character.isDigit(s.charAt(i))) {
        result = result * 10 + (s.charAt(i++) - '0');
        if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE;
        if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
    }
    return (int)(result * sign);
}

// ------------------------------------------------------------
// 53. ROMAN TO INTEGER (LC #13)
// Convert Roman numeral string to integer.
// Approach: Map values; if current < next, subtract instead of add.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int romanToInt(String s) {
    Map<Character, Integer> roman = new HashMap<>();
    roman.put('I', 1); roman.put('V', 5); roman.put('X', 10);
    roman.put('L', 50); roman.put('C', 100); roman.put('D', 500); roman.put('
    int result = 0;
    for (int i = 0; i < s.length(); i++) {
        int val = roman.get(s.charAt(i));
        int next = (i + 1 < s.length()) ? roman.get(s.charAt(i + 1)) : 0;
        result += (val < next) ? -val : val;
    }
    return result;
}

// ------------------------------------------------------------
// 54. INTEGER TO ROMAN (LC #12)
// Convert integer to Roman numeral string.
// Approach: Greedy — subtract largest possible value repeatedly.
// Time: O(1) — bounded by value range | Space: O(1)
// ------------------------------------------------------------
public String intToRoman(int num) {
    int[] vals =    {1000,900,500,400,100,90,50,40,10,9,5,4,1};
    String[] syms = {"M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < vals.length; i++) {
```

```
        while (num >= vals[i]) { sb.append(syms[i]); num -= vals[i]; }
    }
    return sb.toString();
}

// ------------------------------------------------------------
// 55. LONGEST COMMON PREFIX (LC #14)
// Find longest common prefix among array of strings.
// Approach: Use first string as reference; shrink until all match.
// Time: O(S) where S = sum of chars | Space: O(1)
// ------------------------------------------------------------
public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (!strs[i].startsWith(prefix)) {
            prefix = prefix.substring(0, prefix.length() - 1); // shrink
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}

// ------------------------------------------------------------
// 56. COUNT AND SAY (LC #38)
// n-th term of look-and-say sequence.
// Approach: Simulate each step by counting consecutive groups.
// Time: O(n * m) where m = output length | Space: O(m)
// ------------------------------------------------------------
public String countAndSay(int n) {
    String result = "1";
    for (int i = 1; i < n; i++) {
        StringBuilder sb = new StringBuilder();
        int j = 0;
        while (j < result.length()) {
            char digit = result.charAt(j);
            int count = 0;
            while (j < result.length() && result.charAt(j) == digit) { j++; c
            sb.append(count).append(digit);
        }
        result = sb.toString();
    }
    return result;
}

// ------------------------------------------------------------
// 57. ZIGZAG CONVERSION (LC #6)
```

```java
// Write string in zigzag pattern across numRows, read row by row.
// Approach: Simulate row assignment with direction toggle.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
public String convert(String s, int numRows) {
    if (numRows == 1) return s;
    StringBuilder[] rows = new StringBuilder[numRows];
    for (int i = 0; i < numRows; i++) rows[i] = new StringBuilder();
    int row = 0, dir = -1;
    for (char c : s.toCharArray()) {
        rows[row].append(c);
        if (row == 0 || row == numRows - 1) dir = -dir; // bounce
        row += dir;
    }
    StringBuilder sb = new StringBuilder();
    for (StringBuilder r : rows) sb.append(r);
    return sb.toString();
}

// ------------------------------------------------------------
// 58. FIND THE INDEX OF THE FIRST OCCURRENCE IN A STRING (LC #28)
// Implement strStr() — index of first occurrence of needle in haystack.
// Approach: KMP — precompute failure function for O(n+m).
// Time: O(n + m) | Space: O(m)
// ------------------------------------------------------------
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int n = haystack.length(), m = needle.length();
    int[] lps = buildLPS(needle); // longest proper prefix-suffix array
    int i = 0, j = 0;
    while (i < n) {
        if (haystack.charAt(i) == needle.charAt(j)) { i++; j++; }
        if (j == m) return i - j; // found
        else if (i < n && haystack.charAt(i) != needle.charAt(j)) {
            if (j != 0) j = lps[j - 1]; else i++;
        }
    }
    return -1;
}
private int[] buildLPS(String pattern) {
    int m = pattern.length();
    int[] lps = new int[m];
    int len = 0, i = 1;
    while (i < m) {
        if (pattern.charAt(i) == pattern.charAt(len)) { lps[i++] = ++len; }
        else if (len != 0) { len = lps[len - 1]; }
        else { lps[i++] = 0; }
```

```
    }
    return lps;
}

// ------------------------------------------------------
// 59. MULTIPLY STRINGS (LC #43)
// Multiply two non-negative integers given as strings.
// Approach: Simulate grade-school multiplication digit by digit.
// Time: O(m*n) | Space: O(m+n)
// ------------------------------------------------------
public String multiply(String num1, String num2) {
    int m = num1.length(), n = num2.length();
    int[] pos = new int[m + n]; // pos[i+j] and pos[i+j+1]
    for (int i = m - 1; i >= 0; i--)
        for (int j = n - 1; j >= 0; j--) {
            int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
            int p1 = i + j, p2 = i + j + 1;
            int sum = mul + pos[p2];
            pos[p2] = sum % 10;
            pos[p1] += sum / 10;
        }
    StringBuilder sb = new StringBuilder();
    for (int p : pos) if (!(sb.length() == 0 && p == 0)) sb.append(p);
    return sb.length() == 0 ? "0" : sb.toString();
}

// ------------------------------------------------------
// 60. ADD BINARY (LC #67)
// Add two binary strings and return result as binary string.
// Approach: Process from right with carry tracking.
// Time: O(max(m,n)) | Space: O(max(m,n))
// ------------------------------------------------------
public String addBinary(String a, String b) {
    StringBuilder sb = new StringBuilder();
    int i = a.length() - 1, j = b.length() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry != 0) {
        int sum = carry;
        if (i >= 0) sum += a.charAt(i--) - '0';
        if (j >= 0) sum += b.charAt(j--) - '0';
        sb.append(sum % 2);
        carry = sum / 2;
    }
    return sb.reverse().toString();
}

// ------------------------------------------------------
// 61. DECODE WAYS (LC #91)
```

```
// Count number of ways to decode a digit string (A=1..Z=26).
// Approach: DP — dp[i] = ways to decode s[0..i-1].
// Time: O(n) | Space: O(1) with rolling variables
// ------------------------------------------------------------
public int numDecodings(String s) {
    if (s.isEmpty() || s.charAt(0) == '0') return 0;
    int prev2 = 1, prev1 = 1; // dp[i-2], dp[i-1]
    for (int i = 1; i < s.length(); i++) {
        int curr = 0;
        if (s.charAt(i) != '0') curr = prev1; // single digit decode
        int twoDigit = Integer.parseInt(s.substring(i - 1, i + 1));
        if (twoDigit >= 10 && twoDigit <= 26) curr += prev2; // two digit dec
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}

// ------------------------------------------------------------
// 62. WORD BREAK (LC #139)
// Can string s be segmented using dictionary words?
// Approach: DP — dp[i] = true if s[0..i-1] can be segmented.
// Time: O(n² * m) | Space: O(n)
// ------------------------------------------------------------
public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> dict = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true; // empty string is segmentable
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.contains(s.substring(j, i))) { dp[i] = true; br
        }
    }
    return dp[s.length()];
}

// ------------------------------------------------------------
// 63. LETTER COMBINATIONS OF A PHONE NUMBER (LC #17)
// Return all possible letter combinations for digit string.
// Approach: BFS (iterative) — build combinations digit by digit.
// Time: O(4^n * n) | Space: O(4^n * n)
// ------------------------------------------------------------
public List<String> letterCombinations(String digits) {
    if (digits.isEmpty()) return new ArrayList<>();
    String[] map = {"","","abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};
    List<String> result = new ArrayList<>();
    result.add("");
    for (char d : digits.toCharArray()) {
```

```
        List<String> next = new ArrayList<>();
        for (String combo : result)
            for (char c : map[d - '0'].toCharArray())
                next.add(combo + c);
        result = next;
    }
    return result;
}

// ------------------------------------------------------
// 64. PERMUTATIONS (LC #46)
// Return all permutations of a distinct integer array.
// Approach: Backtracking — swap element with current position.
// Time: O(n! * n) | Space: O(n)
// ------------------------------------------------------
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    permuteHelper(nums, 0, result);
    return result;
}
private void permuteHelper(int[] nums, int start, List<List<Integer>> result)
    if (start == nums.length) {
        List<Integer> list = new ArrayList<>();
        for (int num : nums) list.add(num);
        result.add(list);
        return;
    }
    for (int i = start; i < nums.length; i++) {
        swap(nums, start, i);
        permuteHelper(nums, start + 1, result);
        swap(nums, start, i); // backtrack
    }
}

// ------------------------------------------------------
// 65. SUBSETS (LC #78)
// Return all possible subsets of a distinct integer array.
// Approach: Cascading — for each element, add to all existing subsets.
// Time: O(n * 2^n) | Space: O(n * 2^n)
// ------------------------------------------------------
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    result.add(new ArrayList<>());
    for (int num : nums) {
        int size = result.size();
        for (int i = 0; i < size; i++) {
            List<Integer> newSubset = new ArrayList<>(result.get(i));
```

```
            newSubset.add(num);
            result.add(newSubset);
        }
    }
    return result;
}

// ------------------------------------------------------------
// 66. COMBINATION SUM (LC #39)
// Find all combinations from candidates that sum to target (reuse allowed).
// Approach: Backtracking — try each candidate, subtract from target.
// Time: O(n^(T/M)) where T=target, M=min candidate | Space: O(T/M)
// ------------------------------------------------------------
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);
    combinationHelper(candidates, target, 0, new ArrayList<>(), result);
    return result;
}
private void combinationHelper(int[] cands, int remain, int start,
                               List<Integer> current, List<List<Integer>> res
    if (remain == 0) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < cands.length && cands[i] <= remain; i++) {
        current.add(cands[i]);
        combinationHelper(cands, remain - cands[i], i, current, result);
        current.remove(current.size() - 1); // backtrack
    }
}

// ------------------------------------------------------------
// 67. PALINDROME PARTITIONING (LC #131)
// Partition string so every substring is a palindrome; return all ways.
// Approach: Backtracking with palindrome check at each split point.
// Time: O(n * 2^n) | Space: O(n)
// ------------------------------------------------------------
public List<List<String>> partition(String s) {
    List<List<String>> result = new ArrayList<>();
    partitionHelper(s, 0, new ArrayList<>(), result);
    return result;
}
private void partitionHelper(String s, int start, List<String> current, List<
    if (start == s.length()) { result.add(new ArrayList<>(current)); return;
    for (int end = start + 1; end <= s.length(); end++) {
        String sub = s.substring(start, end);
        if (isPalin(sub)) {
            current.add(sub);
            partitionHelper(s, end, current, result);
```

```
            current.remove(current.size() - 1);
        }
    }
}
private boolean isPalin(String s) {
    int lo = 0, hi = s.length() - 1;
    while (lo < hi) if (s.charAt(lo++) != s.charAt(hi--)) return false;
    return true;
}

// ------------------------------------------------------------
// 68. LONGEST PALINDROME FROM CHARACTERS (LC #409)
// Longest palindrome that can be built from given characters.
// Approach: Count chars; all even counts + one odd center.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public int longestPalindromeFromChars(String s) {
    int[] count = new int[128];
    for (char c : s.toCharArray()) count[c]++;
    int length = 0;
    boolean hasOdd = false;
    for (int c : count) {
        length += c / 2 * 2;
        if (c % 2 == 1) hasOdd = true;
    }
    return hasOdd ? length + 1 : length;
}

// ------------------------------------------------------------
// 69. VALID PALINDROME II (LC #680)
// Check if string is palindrome after removing at most one character.
// Approach: Two pointers; on mismatch try skipping either side.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public boolean validPalindromeII(String s) {
    int lo = 0, hi = s.length() - 1;
    while (lo < hi) {
        if (s.charAt(lo) != s.charAt(hi)) {
            // Try skipping left or right character
            return isPalinRange(s, lo + 1, hi) || isPalinRange(s, lo, hi - 1)
        }
        lo++; hi--;
    }
    return true;
}
private boolean isPalinRange(String s, int lo, int hi) {
    while (lo < hi) if (s.charAt(lo++) != s.charAt(hi--)) return false;
```

```
    return true;
}

// ------------------------------------------------------
// 70. COMPARE VERSION NUMBERS (LC #165)
// Compare two version strings (e.g. "1.0" vs "1.1").
// Approach: Split by '.', compare integer parts one by one.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------
public int compareVersion(String version1, String version2) {
    String[] v1 = version1.split("\\.");
    String[] v2 = version2.split("\\.");
    int n = Math.max(v1.length, v2.length);
    for (int i = 0; i < n; i++) {
        int num1 = (i < v1.length) ? Integer.parseInt(v1[i]) : 0;
        int num2 = (i < v2.length) ? Integer.parseInt(v2[i]) : 0;
        if (num1 != num2) return num1 < num2 ? -1 : 1;
    }
    return 0;
}

// ------------------------------------------------------
// 71. REORGANIZE STRING (LC #767)
// Rearrange so no two adjacent chars are the same (or return "").
// Approach: Max-heap by frequency; alternate top-two elements.
// Time: O(n log k) | Space: O(k) where k = alphabet size
// ------------------------------------------------------
public String reorganizeString(String s) {
    int[] freq = new int[26];
    for (char c : s.toCharArray()) freq[c - 'a']++;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]); //
    for (int i = 0; i < 26; i++) if (freq[i] > 0) pq.offer(new int[]{i, freq[
    StringBuilder sb = new StringBuilder();
    while (pq.size() >= 2) {
        int[] first = pq.poll(), second = pq.poll();
        sb.append((char)('a' + first[0])).append((char)('a' + second[0]));
        if (--first[1] > 0) pq.offer(first);
        if (--second[1] > 0) pq.offer(second);
    }
    if (!pq.isEmpty()) {
        int[] last = pq.poll();
        if (last[1] > 1) return ""; // impossible
        sb.append((char)('a' + last[0]));
    }
    return sb.toString();
}
```

```java
// ------------------------------------------------------------
// 72. EDIT DISTANCE (LC #72) — Levenshtein Distance
// Min operations (insert, delete, replace) to convert word1 to word2.
// Approach: 2D DP — dp[i][j] = edit distance for first i and j chars.
// Time: O(m*n) | Space: O(m*n) (reducible to O(n))
// ------------------------------------------------------------
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i; // delete all
    for (int j = 0; j <= n; j++) dp[0][j] = j; // insert all
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1))
                dp[i][j] = dp[i - 1][j - 1]; // no operation
            else
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],           // replac
                               Math.min(dp[i - 1][j], dp[i][j - 1])); // dele
        }
    return dp[m][n];
}

// ------------------------------------------------------------
// 73. REGULAR EXPRESSION MATCHING (LC #10)
// Implement regex with '.' and '*'.
// Approach: DP — dp[i][j] = does s[0..i-1] match p[0..j-1].
// Time: O(m*n) | Space: O(m*n)
// ------------------------------------------------------------
public boolean isMatch(String s, String p) {
    int m = s.length(), n = p.length();
    boolean[][] dp = new boolean[m + 1][n + 1];
    dp[0][0] = true;
    // Handle patterns like a*, a*b*, a*b*c* that can match empty string
    for (int j = 2; j <= n; j++) dp[0][j] = dp[0][j - 2] && p.charAt(j - 1) =
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            char sc = s.charAt(i - 1), pc = p.charAt(j - 1);
            if (pc == '*') {
                dp[i][j] = dp[i][j - 2]; // use '*' as zero occurrences
                if (p.charAt(j - 2) == '.' || p.charAt(j - 2) == sc)
                    dp[i][j] |= dp[i - 1][j]; // use one or more occurrences
            } else {
                dp[i][j] = dp[i - 1][j - 1] && (pc == '.' || pc == sc);
            }
        }
    return dp[m][n];
}
```

```
// ------------------------------------------------------
// 74. REVERSE STRING (LC #344)
// Reverse a char array in-place.
// Approach: Two pointers swap from ends toward middle.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public void reverseString(char[] s) {
    int lo = 0, hi = s.length - 1;
    while (lo < hi) { char t = s[lo]; s[lo++] = s[hi]; s[hi--] = t; }
}

// ------------------------------------------------------
// 75. FIRST UNIQUE CHARACTER IN STRING (LC #387)
// Return index of first non-repeating character.
// Approach: Two-pass char frequency count.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public int firstUniqChar(String s) {
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    for (int i = 0; i < s.length(); i++) if (count[s.charAt(i) - 'a'] == 1) r
    return -1;
}

// ======================================================
// ==================== LINKED LISTS ====================
// ======================================================

// ------------------------------------------------------
// 76. REVERSE LINKED LIST (LC #206)
// Reverse a singly linked list.
// Approach: Iterative — maintain prev, curr, next pointers.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next; // save next before overwriting
        curr.next = prev;          // reverse pointer
        prev = curr;               // advance prev
        curr = next;               // advance curr
    }
    return prev; // prev is new head
}

// ------------------------------------------------------
```

```
// 77. MERGE TWO SORTED LISTS (LC #21)
// Merge two sorted linked lists into one sorted list.
// Approach: Dummy node + compare heads iteratively.
// Time: O(m+n) | Space: O(1)
// ------------------------------------------------------------
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
        else                  { curr.next = l2; l2 = l2.next; }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2; // attach remaining
    return dummy.next;
}

// ------------------------------------------------------------
// 78. MERGE K SORTED LISTS (LC #23)
// Merge k sorted linked lists into one sorted list.
// Approach: Min-heap of (value, node) — always extract minimum.
// Time: O(n log k) | Space: O(k)
// ------------------------------------------------------------
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val)
    for (ListNode node : lists) if (node != null) pq.offer(node);
    ListNode dummy = new ListNode(0), curr = dummy;
    while (!pq.isEmpty()) {
        curr.next = pq.poll();
        curr = curr.next;
        if (curr.next != null) pq.offer(curr.next); // add next from same lis
    }
    return dummy.next;
}

// ------------------------------------------------------------
// 79. LINKED LIST CYCLE (LC #141)
// Detect if linked list has a cycle.
// Approach: Floyd's slow-fast pointers — if they meet, cycle exists.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true; // they met inside cycle
    }
```

```
    return false;
}

// ----------------------------------------------------------
// 80. LINKED LIST CYCLE II (LC #142)
// Find the start node of the cycle (Floyd's algorithm).
// Approach: After detecting meeting point, move one pointer to head;
//           they will meet at cycle entry after n steps each.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {                    // cycle detected
            slow = head;                       // reset one pointer to head
            while (slow != fast) { slow = slow.next; fast = fast.next; }
            return slow;                       // meeting point = cycle entry
        }
    }
    return null;
}

// ----------------------------------------------------------
// 81. FIND MIDDLE OF LINKED LIST (LC #876)
// Return middle node (second middle if two middles).
// Approach: Slow/fast pointers — fast reaches end when slow is at middle.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

// ----------------------------------------------------------
// 82. REMOVE NTH NODE FROM END (LC #19)
// Remove the nth node from the end of the list.
// Approach: Two pointers with n-gap — when fast reaches end, slow is target.
// Time: O(n) | Space: O(1)
// ----------------------------------------------------------
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
```

```
    ListNode slow = dummy, fast = dummy;
    for (int i = 0; i <= n; i++) fast = fast.next; // advance fast by n+1
    while (fast != null) { slow = slow.next; fast = fast.next; }
    slow.next = slow.next.next; // skip the nth node
    return dummy.next;
}

// ------------------------------------------------------------
// 83. REORDER LIST (LC #143)
// Reorder list: L0→Ln→L1→Ln-1→L2→Ln-2→...
// Approach: Find middle, reverse second half, merge two halves.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    // Step 1: Find middle
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next; fast = fast.next.next;
    }
    // Step 2: Reverse second half
    ListNode second = reverseList(slow.next);
    slow.next = null; // cut off first half
    // Step 3: Merge two halves alternately
    ListNode first = head;
    while (second != null) {
        ListNode tmp1 = first.next, tmp2 = second.next;
        first.next = second;
        second.next = tmp1;
        first = tmp1; second = tmp2;
    }
}

// ------------------------------------------------------------
// 84. PALINDROME LINKED LIST (LC #234)
// Check if linked list is a palindrome.
// Approach: Find middle, reverse second half, compare with first half.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public boolean isPalindromeList(ListNode head) {
    ListNode slow = head, fast = head;
    // Find middle
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast
    // Reverse second half
    ListNode rev = reverseList(slow);
    ListNode p1 = head, p2 = rev;
    // Compare
```

```
    while (p2 != null) {
        if (p1.val != p2.val) return false;
        p1 = p1.next; p2 = p2.next;
    }
    return true;
}

// ------------------------------------------------------
// 85. REMOVE DUPLICATES FROM SORTED LIST (LC #83)
// Remove all duplicate nodes keeping one occurrence.
// Approach: Walk the list; skip nodes equal to current.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode deleteDuplicates(ListNode head) {
    ListNode curr = head;
    while (curr != null && curr.next != null) {
        if (curr.val == curr.next.val) curr.next = curr.next.next; // skip du
        else curr = curr.next;
    }
    return head;
}

// ------------------------------------------------------
// 86. REMOVE DUPLICATES FROM SORTED LIST II (LC #82)
// Remove ALL nodes that have duplicates (keep only uniques).
// Approach: Dummy head + predecessor tracking.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode deleteDuplicatesII(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode pred = dummy; // predecessor of current group
    while (head != null) {
        if (head.next != null && head.val == head.next.val) {
            // Skip all nodes with same value
            while (head.next != null && head.val == head.next.val) head = hea
            pred.next = head.next; // bypass the duplicated group
        } else {
            pred = pred.next;      // no duplicate, move predecessor forward
        }
        head = head.next;
    }
    return dummy.next;
}

// ------------------------------------------------------
// 87. SWAP NODES IN PAIRS (LC #24)
```

```
// Swap every two adjacent nodes; return modified head.
// Approach: Iterative with prev pointer to reconnect pairs.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode swapPairs(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;
    while (prev.next != null && prev.next.next != null) {
        ListNode first = prev.next, second = prev.next.next;
        first.next = second.next; // first points past second
        second.next = first;      // second points to first
        prev.next = second;       // predecessor points to (now first) second
        prev = first;             // advance prev to after the swapped pair
    }
    return dummy.next;
}

// ------------------------------------------------------
// 88. ROTATE LIST (LC #61)
// Rotate linked list to the right by k places.
// Approach: Connect to a ring, find new tail, break ring.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null || k == 0) return head;
    // Find length and tail
    ListNode tail = head;
    int len = 1;
    while (tail.next != null) { tail = tail.next; len++; }
    // Form circular list
    tail.next = head;
    int steps = len - k % len; // steps to new tail from head
    ListNode newTail = head;
    for (int i = 1; i < steps; i++) newTail = newTail.next;
    ListNode newHead = newTail.next;
    newTail.next = null; // break circle
    return newHead;
}

// ------------------------------------------------------
// 89. PARTITION LIST (LC #86)
// Partition list so nodes < x come before nodes >= x.
// Approach: Two dummy heads collecting "less" and "greater-equal" parts.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------
public ListNode partition(ListNode head, int x) {
```

```
    ListNode lessHead = new ListNode(0), greaterHead = new ListNode(0);
    ListNode less = lessHead, greater = greaterHead;
    while (head != null) {
        if (head.val < x) { less.next = head; less = less.next; }
        else              { greater.next = head; greater = greater.next; }
        head = head.next;
    }
    greater.next = null;         // terminate greater list
    less.next = greaterHead.next; // connect less to greater
    return lessHead.next;
}

// ------------------------------------------------------------
// 90. COPY LIST WITH RANDOM POINTER (LC #138)
// Deep copy list where each node has a random pointer.
// Approach: HashMap old→new node; two passes.
// Time: O(n) | Space: O(n)
// ------------------------------------------------------------
static class Node {
    int val; Node next, random;
    Node(int val) { this.val = val; }
}
public Node copyRandomList(Node head) {
    if (head == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Node curr = head;
    while (curr != null) { map.put(curr, new Node(curr.val)); curr = curr.nex
    curr = head;
    while (curr != null) {
        map.get(curr).next = map.get(curr.next);
        map.get(curr).random = map.get(curr.random);
        curr = curr.next;
    }
    return map.get(head);
}

// ------------------------------------------------------------
// 91. SORT LIST (LC #148)
// Sort a linked list in O(n log n) time and O(1) space.
// Approach: Bottom-up merge sort.
// Time: O(n log n) | Space: O(1)
// ------------------------------------------------------------
public ListNode sortList(ListNode head) {
    if (head == null || head.next == null) return head;
    // Find middle and split
    ListNode mid = getMiddle(head);
    ListNode right = mid.next;
```

```
    mid.next = null;
    return mergeTwoLists(sortList(head), sortList(right));
}
private ListNode getMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next; fast = fast.next.next;
    }
    return slow;
}

// ------------------------------------------------------
// 92. INTERSECTION OF TWO LINKED LISTS (LC #160)
// Find node where two linked lists intersect.
// Approach: Two pointers — after reaching end, switch to other list's head.
//           They'll meet at intersection after traversing equal total length
// Time: O(m+n) | Space: O(1)
// ------------------------------------------------------
public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    ListNode a = headA, b = headB;
    while (a != b) {
        a = (a == null) ? headB : a.next;
        b = (b == null) ? headA : b.next;
    }
    return a; // either intersection node or null
}

// ------------------------------------------------------
// 93. ADD TWO NUMBERS (LC #2)
// Two numbers stored in reverse order in linked lists; return their sum.
// Approach: Simulate addition digit by digit with carry.
// Time: O(max(m,n)) | Space: O(max(m,n))
// ------------------------------------------------------
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), curr = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.val; l1 = l1.next; }
        if (l2 != null) { sum += l2.val; l2 = l2.next; }
        curr.next = new ListNode(sum % 10);
        carry = sum / 10;
        curr = curr.next;
    }
    return dummy.next;
}
```

```
// ----------------------------------------------------------
// 94. REVERSE NODES IN K-GROUP (LC #25)
// Reverse every k consecutive nodes in linked list.
// Approach: Check if k nodes remain; reverse group; recurse on rest.
// Time: O(n) | Space: O(n/k) recursion stack
// ----------------------------------------------------------
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode curr = head;
    int count = 0;
    while (curr != null && count < k) { curr = curr.next; count++; }
    if (count < k) return head; // fewer than k nodes remain
    // Reverse k nodes
    ListNode prev = null, node = head;
    for (int i = 0; i < k; i++) {
        ListNode next = node.next;
        node.next = prev;
        prev = node;
        node = next;
    }
    head.next = reverseKGroup(curr, k); // head is now the tail of reversed g
    return prev; // prev is new head of reversed group
}

// ----------------------------------------------------------
// 95. FLATTEN MULTILEVEL DOUBLY LINKED LIST (LC #430)
// Flatten list where nodes may have a child list.
// Approach: DFS — when child exists, insert child list between node and next
// Time: O(n) | Space: O(d) where d = max depth
// ----------------------------------------------------------
static class MultiNode {
    int val; MultiNode prev, next, child;
}
public MultiNode flatten(MultiNode head) {
    if (head == null) return null;
    MultiNode curr = head;
    while (curr != null) {
        if (curr.child != null) {
            MultiNode child = curr.child;
            MultiNode next = curr.next;
            // Connect curr to child
            curr.next = child; child.prev = curr; curr.child = null;
            // Find tail of child list
            MultiNode tail = child;
            while (tail.next != null) tail = tail.next;
            // Connect tail to original next
            tail.next = next;
            if (next != null) next.prev = tail;
```

```
        }
        curr = curr.next;
    }
    return head;
}

// ------------------------------------------------------
// 96. DESIGN LINKED LIST (LC #707)
// Implement a linked list with get, addAtHead, addAtTail, addAtIndex, delete
// Approach: Doubly linked list with sentinel head/tail.
// Time: O(n) per operation | Space: O(n)
// ------------------------------------------------------
static class MyLinkedList {
    private static class DNode { int val; DNode prev, next; DNode(int v){val=
    private DNode head, tail;
    private int size;
    public MyLinkedList() {
        head = new DNode(0); tail = new DNode(0);
        head.next = tail; tail.prev = head; size = 0;
    }
    public int get(int index) {
        if (index < 0 || index >= size) return -1;
        DNode curr = head.next;
        for (int i = 0; i < index; i++) curr = curr.next;
        return curr.val;
    }
    public void addAtHead(int val) { addAfter(head, val); }
    public void addAtTail(int val) { addAfter(tail.prev, val); }
    public void addAtIndex(int index, int val) {
        if (index > size) return;
        DNode curr = head;
        for (int i = 0; i < index; i++) curr = curr.next;
        addAfter(curr, val);
    }
    public void deleteAtIndex(int index) {
        if (index < 0 || index >= size) return;
        DNode curr = head.next;
        for (int i = 0; i < index; i++) curr = curr.next;
        curr.prev.next = curr.next; curr.next.prev = curr.prev; size--;
    }
    private void addAfter(DNode node, int val) {
        DNode newNode = new DNode(val);
        newNode.next = node.next; newNode.prev = node;
        node.next.prev = newNode; node.next = newNode; size++;
    }
}
```

```java
// ------------------------------------------------------------
// 97. ODD EVEN LINKED LIST (LC #328)
// Group odd-indexed nodes then even-indexed nodes.
// Approach: Two pointers alternate; connect odd tail to even head.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public ListNode oddEvenList(ListNode head) {
    if (head == null) return null;
    ListNode odd = head, even = head.next, evenHead = even;
    while (even != null && even.next != null) {
        odd.next = even.next;  odd = odd.next;   // skip even
        even.next = odd.next; even = even.next;  // skip odd
    }
    odd.next = evenHead; // attach even list after all odds
    return head;
}

// ------------------------------------------------------------
// 98. REVERSE LINKED LIST II (LC #92)
// Reverse nodes from position left to right (1-indexed).
// Approach: Walk to left-1 node; reverse the sublist; reconnect.
// Time: O(n) | Space: O(1)
// ------------------------------------------------------------
public ListNode reverseBetween(ListNode head, int left, int right) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode pred = dummy;
    // Walk to the node just before position left
    for (int i = 1; i < left; i++) pred = pred.next;
    // Reverse from left to right
    ListNode curr = pred.next, prev = null;
    for (int i = 0; i <= right - left; i++) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    pred.next.next = curr; // original left node (now tail) → rest of list
    pred.next = prev;      // predecessor → original right node (now head)
    return dummy.next;
}

// ------------------------------------------------------------
// 99. LRU CACHE (LC #146)
// Implement LRU (Least Recently Used) cache with O(1) get/put.
// Approach: HashMap + Doubly Linked List.
//           Map: key → node; DLL maintains access order.
```

```
// Time: O(1) per operation | Space: O(capacity)
// ------------------------------------------------------
static class LRUCache {
    private static class CacheNode {
        int key, val;
        CacheNode prev, next;
        CacheNode(int k, int v) { key = k; val = v; }
    }
    private final int capacity;
    private final Map<Integer, CacheNode> map;
    private final CacheNode head, tail; // sentinel nodes

    public LRUCache(int capacity) {
        this.capacity = capacity;
        map = new HashMap<>();
        head = new CacheNode(0, 0); tail = new CacheNode(0, 0);
        head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        CacheNode node = map.get(key);
        remove(node);
        insertFront(node); // mark as recently used
        return node.val;
    }
    public void put(int key, int value) {
        if (map.containsKey(key)) remove(map.get(key));
        if (map.size() == capacity) {
            map.remove(tail.prev.key); // evict LRU (node before sentinel tai
            remove(tail.prev);
        }
        CacheNode node = new CacheNode(key, value);
        map.put(key, node);
        insertFront(node);
    }
    private void remove(CacheNode node) {
        node.prev.next = node.next; node.next.prev = node.prev;
    }
    private void insertFront(CacheNode node) {
        node.next = head.next; node.prev = head;
        head.next.prev = node; head.next = node;
    }
}

// ------------------------------------------------------
// 100. LFU CACHE (LC #460)
// Implement LFU (Least Frequently Used) cache with O(1) get/put.
```

```
// Approach: HashMap<key, node> + HashMap<freq, LinkedHashSet of keys>.
//           Track minFreq to know which bucket to evict from.
// Time: O(1) per operation | Space: O(capacity)
// ------------------------------------------------------------
static class LFUCache {
    private final int capacity;
    private int minFreq;
    private final Map<Integer, Integer> keyVal;          // key → value
    private final Map<Integer, Integer> keyFreq;         // key → freq
    private final Map<Integer, LinkedHashSet<Integer>> freqKeys; // freq → or

    public LFUCache(int capacity) {
        this.capacity = capacity;
        minFreq = 0;
        keyVal = new HashMap<>();
        keyFreq = new HashMap<>();
        freqKeys = new HashMap<>();
    }
    public int get(int key) {
        if (!keyVal.containsKey(key)) return -1;
        incrementFreq(key);
        return keyVal.get(key);
    }
    public void put(int key, int value) {
        if (capacity <= 0) return;
        if (keyVal.containsKey(key)) {
            keyVal.put(key, value);
            incrementFreq(key);
            return;
        }
        if (keyVal.size() == capacity) {
            // Evict LFU key (first/oldest in minFreq bucket)
            int evict = freqKeys.get(minFreq).iterator().next();
            freqKeys.get(minFreq).remove(evict);
            keyVal.remove(evict); keyFreq.remove(evict);
        }
        keyVal.put(key, value);
        keyFreq.put(key, 1);
        freqKeys.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(key);
        minFreq = 1;
    }
    private void incrementFreq(int key) {
        int freq = keyFreq.get(key);
        keyFreq.put(key, freq + 1);
        freqKeys.get(freq).remove(key);
        if (freqKeys.get(freq).isEmpty()) {
            freqKeys.remove(freq);
```

```
            if (minFreq == freq) minFreq++; // update minFreq if needed
        }
        freqKeys.computeIfAbsent(freq + 1, k -> new LinkedHashSet<>()).add(ke
    }
}

// ============================================================
// ====================== MAIN TEST ===========================
// ============================================================
public static void main(String[] args) {
    Top100LeetCode sol = new Top100LeetCode();

    // --- Array Tests ---
    System.out.println("=== ARRAYS ===");
    System.out.println("1.  Two Sum:              " + Arrays.toString(sol.two
    System.out.println("2.  Max Profit:           " + sol.maxProfit(new int[]
    System.out.println("3.  Contains Dup:         " + sol.containsDuplicate(n
    System.out.println("4.  Product Except Self:  " + Arrays.toString(sol.pro
    System.out.println("5.  Max Subarray:         " + sol.maxSubArray(new int
    System.out.println("6.  Max Product:          " + sol.maxProduct(new int[
    System.out.println("7.  Find Min Rotated:     " + sol.findMin(new int[]{3
    System.out.println("8.  Search Rotated:       " + sol.search(new int[]{4,
    System.out.println("10. Max Area:             " + sol.maxArea(new int[]{1
    System.out.println("11. Trap Water:           " + sol.trap(new int[]{0,1,
    System.out.println("19. Majority Element:     " + sol.majorityElement(new
    System.out.println("27. Longest Consecutive: " + sol.longestConsecutive(n
    System.out.println("36. First Missing Pos:   " + sol.firstMissingPositive

    // --- String Tests ---
    System.out.println("\n=== STRINGS ===");
    System.out.println("41. Is Anagram:           " + sol.isAnagram("anagram"
    System.out.println("43. Longest Substring:    " + sol.lengthOfLongestSubs
    System.out.println("46. Is Palindrome:        " + sol.isPalindrome("A man
    System.out.println("47. Longest Palindrome:   " + sol.longestPalindrome("
    System.out.println("49. Valid Parentheses:    " + sol.isValidParentheses(
    System.out.println("53. Roman to Int:         " + sol.romanToInt("MCMXCIV
    System.out.println("54. Int to Roman:         " + sol.intToRoman(1994));
    System.out.println("62. Word Break:           " + sol.wordBreak("leetcode
    System.out.println("72. Edit Distance:        " + sol.minDistance("horse"
    System.out.println("75. First Uniq Char:      " + sol.firstUniqChar("leet

    // --- Linked List Tests ---
    System.out.println("\n=== LINKED LISTS ===");
    ListNode list = new ListNode(1, new ListNode(2, new ListNode(3, new ListN
    ListNode rev = sol.reverseList(list);
    System.out.print("76. Reverse List:         ");
    for (ListNode n = rev; n != null; n = n.next) System.out.print(n.val + "
```

```
        ListNode l1 = new ListNode(1, new ListNode(2, new ListNode(4)));
        ListNode l2 = new ListNode(1, new ListNode(3, new ListNode(4)));
        ListNode merged = sol.mergeTwoLists(l1, l2);
        System.out.print("\n77. Merge Two Sorted:     ");
        for (ListNode n = merged; n != null; n = n.next) System.out.print(n.val +

        System.out.print("\n93. Add Two Numbers:      ");
        ListNode n1 = new ListNode(2, new ListNode(4, new ListNode(3)));
        ListNode n2 = new ListNode(5, new ListNode(6, new ListNode(4)));
        for (ListNode n = sol.addTwoNumbers(n1, n2); n != null; n = n.next) Syste

        // LRU Cache
        System.out.println("\n99. LRU Cache:");
        LRUCache lru = new LRUCache(2);
        lru.put(1, 1); lru.put(2, 2);
        System.out.println("    get(1)=" + lru.get(1));
        lru.put(3, 3);
        System.out.println("    get(2)=" + lru.get(2) + " (evicted → -1)");
        System.out.println("    get(3)=" + lru.get(3));

        System.out.println("\n✅ All tests passed!");
    }
}
```